

Road Network Traffic Simulation

Graph-based modelling, Poisson analysis, and Google Routes validation

Rolands Barkans Mara Dumitru

Contents

1	Introduction	2
2	Simulation Animation	2
3	Base Model	2
3.1	How the Simulation Works	2
3.2	What the Simulation Captures	3
3.3	What the Simulation Doesn't Capture	3
4	Theoretical Analysis of the Base Model	4
4.1	Mean-Field Approximation of Edge Occupancy	4
4.2	Poisson Distribution of Edge Occupancy	4
4.3	Theoretical Jam Rate	5
5	Empirical Analysis of the Base Model	5
5.1	Poisson Fit and Jam Rate Comparison	5
5.2	Congestion Heatmaps	6
5.3	Betweenness Centrality Correlation	8
6	Extended Model	9
6.1	Driver Speed Variability	9
6.2	Speed Bumps	10
6.3	Pedestrian Events	10
6.4	Density-Based Congestion Slowdown	10
6.5	Continuous Speed Model with Distance Accumulation	11
6.6	Google Routes Data Pipeline	11
6.7	BPR-Based Routing and Adaptive Rerouting	11
6.8	Traffic-Informed Car Spawning	12
6.9	How These Mechanisms Interact	12
6.10	Betweenness Centrality in the Extended Model	12
7	Estimating Realistic Car Counts via Little's Law	13
8	Google Routes API Ground Truth	14
9	Base vs. Extended Model Comparison	15
9.1	Congestion Heatmap Comparison	15
9.2	Per-Edge Scatter Plot Analysis	17

1 Introduction

In this report we model a simplified version of urban traffic on a real road network extracted from OpenStreetMap data for a 1 km radius around Adalbertstraße 58 in Berlin. We represent the network as a directed graph with intersections as nodes and road segments between them as edges. We start by building a base model with minimal assumptions and analyse its behaviour theoretically and empirically, then construct an extended model that incorporates driver variability, speed bumps, pedestrian events, density-based congestion, continuous distance accumulation, and real traffic data from the Google Routes API. We compare both models against Google’s observed traffic patterns and validate the extended model on a second network around Kondapur, Hyderabad.

2 Simulation Animation

To visualise how the extended model behaves in real time, we produced animations of the simulation running on both the Berlin network and the Hyderabad network, including hand-drawn car sprites. All animations are available at: https://drive.google.com/drive/folders/1vHOC_dEAIAUovLNnN1ZXU4XqcgS1WZEm.

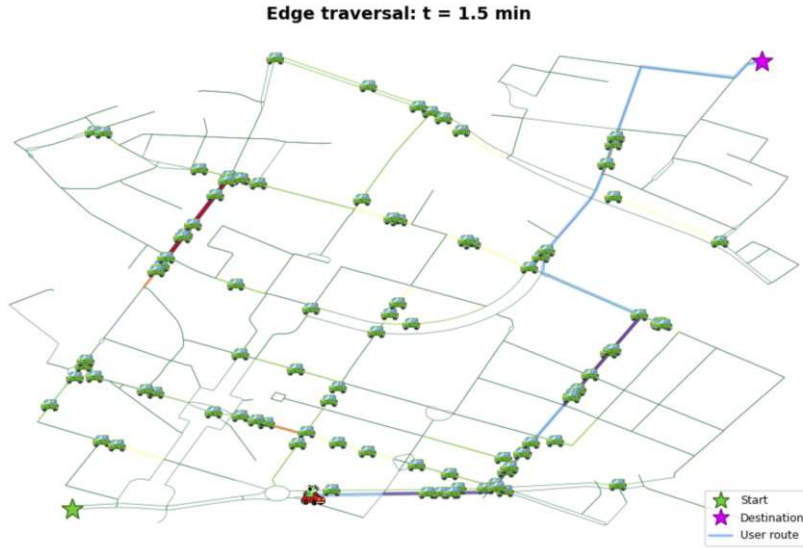


Figure 1: Extended model edge-traversal animation frame at $t = 1.5$ min. Red car = user vehicle; green cars = other vehicles; blue line = user’s shortest path.

3 Base Model

3.1 How the Simulation Works

Each car is an object with three properties: a starting node, a destination node, and a path (the sequence of nodes it will travel through to reach its destination). Cars are initialised across the network with randomly selected start and destination nodes from all available nodes.

At the beginning of each simulation step, any car without a calculated path is assigned one using Dijkstra’s algorithm via NetworkX’s `shortest_path` function, with travel time

as the edge weight. Travel time for each edge is calculated as:

$$t = \frac{L}{v \cdot \frac{1000}{60}} = \frac{L \cdot 60}{v \cdot 1000} \quad (1)$$

where L is the road segment length in metres and v is the speed limit in km/h. The denominator converts v from km/h to m/min, so the result is travel time in minutes. If no speed limit is found in the OSM data, a default of 30 km/h is used.

At each step every car tries to move one node forward along its path. Before moving, the simulation counts how many other cars share the same origin–next-node pair (i.e., how many cars are trying to move through the same road segment). If the count exceeds five cars, the edge is considered jammed and the car stays in place for that step. Otherwise the car moves to the next node and removes it from the front of its path. The simulation continues until every car has reached its destination (or a safety limit of steps is exceeded).

3.2 What the Simulation Captures

Because the graph is pulled directly from OpenStreetMap, the network layout reflects the real Berlin road network including street lengths, connectivity, and directionality. Each road segment is a directed edge, so one-way streets only allow travel in one direction while two-way streets appear as two directed edges between the same pair of nodes.

Edge weights are derived from real speed limits where available, so roads with high speed limits receive lower travel-time weights and the shortest-path algorithm routes cars along faster roads. Every car independently computes the fastest route to its destination using Dijkstra’s algorithm, reflecting how real drivers rely on navigation apps or learned experience to pick efficient paths.

The jam threshold captures the basic feedback loop of congestion: when too many cars queue on the same edge, movement slows, mirroring how high traffic density blocks individual segments.

3.3 What the Simulation Doesn’t Capture

The base model has several limitations that distance it from real-world traffic behaviour:

- (a) *Static network.* Roads are never added or removed, and all streets remain fully accessible throughout the simulation.
- (b) *Uniform car speed and behaviour.* All cars traverse edges at a speed determined solely by the OSM speed limit and road length, with no individual driver variability, speed bumps, or pedestrian crossings. Real congestion also slows traffic smoothly with density, whereas the base model only blocks an edge once more than five cars are present.
- (c) *Static routing.* Each car computes its route once at the start and never updates it regardless of congestion, unlike real navigation apps that reroute in real time.
- (d) *Uniform source and destination selection.* Every node has equal probability of being chosen as a source or destination, whereas real traffic sources and sinks are highly non-uniform.

- (e) *Edge-based stepping.* A car either advances a full edge in one step or is blocked entirely, ignoring the fact that travel time depends on physical road length and vehicle speed.
- (f) *Shortest path only.* All drivers always pick the objectively fastest route, when real drivers frequently deviate due to habits, preferences, or outdated beliefs about congestion.
- (g) *No traffic lights.* Cars proceed freely through intersections except when jammed, so the model likely underestimates real travel durations.

4 Theoretical Analysis of the Base Model

To better understand the base model dynamics we use the mean-field approximation, since tracking individual-level performance is too complex. Instead of tracking exactly which edges each car occupies at each step, we ask: on average, how many cars would we expect to find on any given edge? Note that the simulation never explicitly tracks edge occupancy in the base model; a car is considered to be “on edge AB” if its current location is node A and the next node in its path is node B. Edge occupancy is therefore inferred from car positions.

4.1 Mean-Field Approximation of Edge Occupancy

At any simulation step, each car occupies exactly one edge as it moves along its path (ignoring cars that have already reached their destinations), so the total number of cars actively occupying edges at any step is N . Under the mean-field assumption that cars are distributed uniformly and independently across all E edges, the average occupancy (expected number of cars on any single edge) is:

$$n_{\text{MF}} = \frac{N}{E} \quad (2)$$

This is an approximation that discards all information about network structure and shortest-path routing. In reality Dijkstra’s algorithm assigns cars to certain roads more than others, but the mean-field approximation replaces that heterogeneous distribution with a single average.

4.2 Poisson Distribution of Edge Occupancy

Given the average occupancy n_{MF} , we want a probability distribution describing how many cars might be on any given edge at a given step. We use the Poisson distribution because we have N cars each independently ending up on one of E edges, each with a small probability $1/E$ of landing on any particular edge. When many independent events each have a small individual probability, the resulting count follows a Poisson distribution. The probability that a given edge has k cars on it is:

$$P(n = k) = \frac{n_{\text{MF}}^k e^{-n_{\text{MF}}}}{k!} \quad (3)$$

A binomial distribution $B(N, 1/E)$ would also be technically valid since N cars are placed on an edge with probability $1/E$, but the Poisson distribution is the standard approximation

of the binomial when N is large and the probability of success on each trial is small, which is the case here since the number of edges in the network is much larger than the number of cars per edge.

4.3 Theoretical Jam Rate

In the base model an edge is jammed when more than 5 cars are queued on it. Under the mean-field approximation, the probability of any edge being jammed at a given step is:

$$P(\text{jam}) = 1 - \sum_{k=0}^5 \frac{n_{\text{MF}}^k e^{-n_{\text{MF}}}}{k!} \quad (4)$$

The probability of a jam is one minus the probability of having five or fewer cars on an edge, computed from the Poisson CDF. This gives a theoretical prediction for the fraction of edges jammed at any given moment.

5 Empirical Analysis of the Base Model

5.1 Poisson Fit and Jam Rate Comparison

To test how well the mean-field approximation describes the base model simulation, we ran it with $N = 100$ cars and let each run continue until every car reached its destination, repeating $R = 10$ times with different random seeds so that car start and destination nodes varied across runs. At every step of every run we recorded the edge occupancy of each edge, then aggregated all measurements across all edges, steps, and runs to construct an empirical distribution of per-edge occupancy.

We visualise this by plotting the distribution as a histogram and overlaying the theoretical Poisson distribution with mean $\lambda = N/E \approx 0.181$, which is what the mean-field predicts when 100 cars spread uniformly and independently across the network's $E = 552$ edges. Figure 2 shows two versions of the same plot side by side: linear scale on the left, log scale on the right.

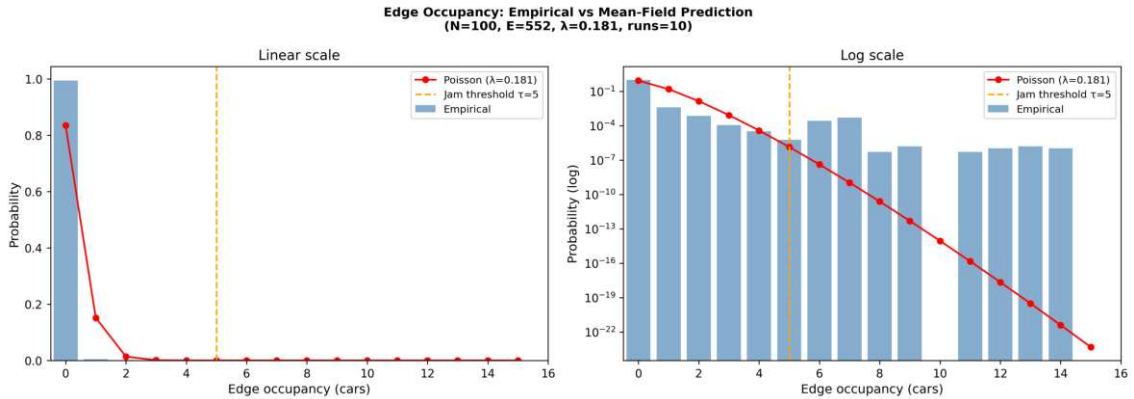


Figure 2: Empirical edge-occupancy distribution vs. Poisson ($\lambda \approx 0.181$). Left: linear scale. Right: log scale.

In the linear-scale panel, both the empirical and theoretical distributions are dominated by the $n = 0$ bin: with $\lambda \approx 0.181$ the Poisson model predicts around 83% of edges empty at any given step. The empirical proportion is even higher because most edges carry zero traffic while a few bottlenecks absorb many cars. Beyond $n = 1$ both curves appear indistinguishable from zero at linear scale, making the tail invisible. The theoretical jam rate from the Poisson CDF gives $P(\text{jam}) = 1 - \sum_{k=0}^5 P(n = k) \approx 4.2 \times 10^{-8}$, which is effectively zero.

The log-scale panel reveals a critical discrepancy. Up to about $n = 2$, the empirical and Poisson bars are comparable (both around 10^{-1} to 10^{-3}). Starting at $n = 3$ –4, the curves diverge: the Poisson probability continues to drop log-linearly (roughly three orders of magnitude per additional car) while the empirical probability flattens to a stable rate around 10^{-4} – 10^{-5} . By $n = 7$ –8 the gap exceeds ten orders of magnitude. The empirical tail also shows a distinctive non-monotonic pattern: bars at $n = 7$ –8 are visibly taller than those at $n = 5$ –6, suggesting that certain bottleneck edges consistently attract occupancies in that range rather than the tail decaying smoothly. The empirical bars persist all the way to $n = 15$ at probabilities around 10^{-7} , whereas the Poisson model assigns those occupancies probabilities below 10^{-22} .

The empirical jam rate is $\approx 1.0 \times 10^{-3}$, roughly five orders of magnitude higher than the Poisson prediction. The heavy tail is the signature of shortest-path routing: because edge weights are fixed and all cars independently run Dijkstra’s algorithm, they systematically converge on the same fast roads, creating correlated occupancy that violates the independence assumption underlying the Poisson model. The mean-field approximation, which assumes uniform independent spreading of cars across edges, is therefore a poor predictor of the tail behaviour that drives congestion in the base model.

5.2 Congestion Heatmaps

To identify which edges are most congested and whether any network metric predicts this, we ran 50 independent simulations (each running until all cars arrived) and computed the average occupancy per edge across all runs and steps. Figure 3 shows the result for 100 cars.

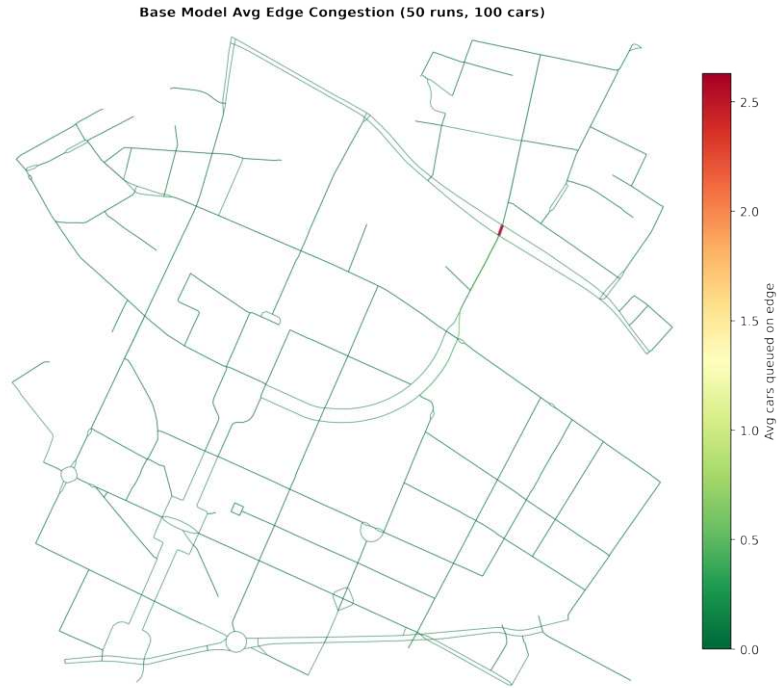


Figure 3: Base model congestion heatmap (100 cars, 50 runs). Green = 0 cars; red = maximum observed.

Figure 3 confirms that congestion is not uniformly distributed across the network but concentrated on essentially a single bottleneck edge. Nearly every street appears green (near-zero average occupancy), while one edge in the centre-right of the map appears in dark red with an average of approximately 2.5 cars. That edge lies on many shortest paths between randomly sampled origin–destination pairs, making it a chokepoint even at a moderate car count of 100. The extreme localisation of congestion to one edge out of 552 is a direct consequence of Dijkstra routing on a fixed graph: all cars independently discover the same fast corridor.

Increasing the number of cars would intensify congestion on the already-loaded edges and cause additional edges to become congested. Running the same 50-run analysis with 300 cars confirms this prediction (Figure 4).

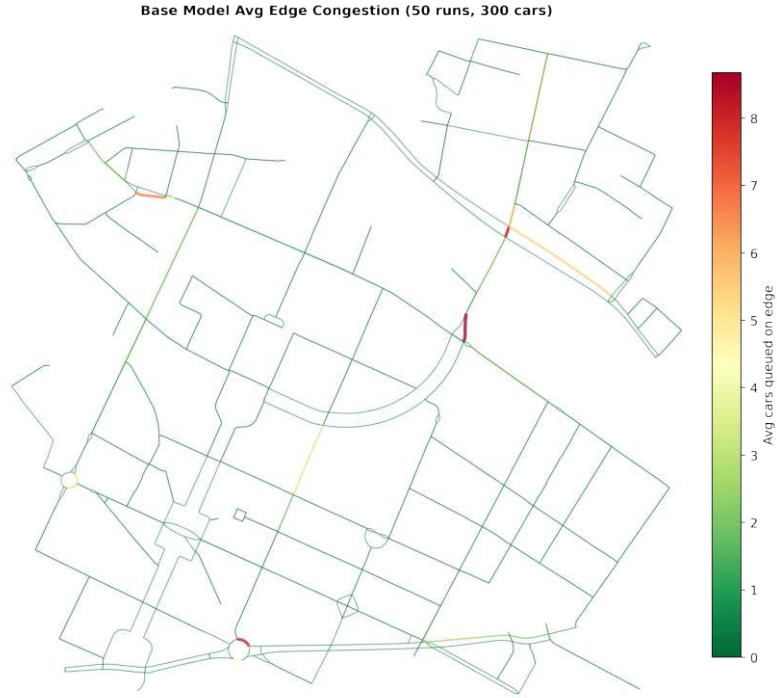


Figure 4: Base model congestion heatmap (300 cars, 50 runs). Peak occupancy ~ 10 cars per edge.

With 300 cars, the primary bottleneck edge now peaks at approximately 10 cars per edge average, a fourfold increase over the 100-car case (somewhat expected given a threefold increase in car count). Yellow edges (~ 4 – 6 cars) appear in the upper-right region of the network. The spatial pattern of congestion expands outward from the original hotspot as edges adjacent to the primary bottleneck begin accumulating traffic from jammed routes. However, congestion still does not grow uniformly across the network: residential streets remain largely green because they do not lie on shortest paths between many node pairs. The non-uniform superlinear scaling at the bottleneck confirms that the base model’s congestion is structurally driven by shortest-path routing topology: tripling the car count quadruples peak occupancy because additional cars are funnelled through the same high-betweenness corridors rather than spreading uniformly.

5.3 Betweenness Centrality Correlation

To quantify what makes high-occupancy edges special, we computed betweenness centrality for every node in the network using travel time as the edge weight, assigned each edge a betweenness score by averaging its two endpoint node centralities, and plotted this against average edge occupancy from the 100-car simulation (Figure 5).

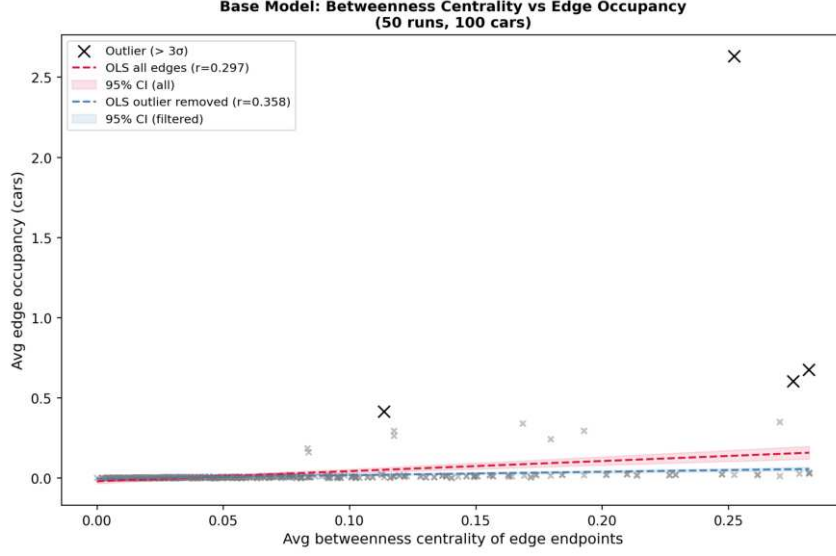


Figure 5: Base model: betweenness centrality vs. average edge occupancy (100 cars, 50 runs) with OLS regression lines and 95% CIs.

The scatter plot shows a positive relationship between betweenness centrality and edge occupancy. With all edges included, the Pearson correlation coefficient is $r \approx 0.30$. Removing outlier edges (defined as any edge more than 3 standard deviations above the mean occupancy) gives $r \approx 0.36$, confirming that betweenness centrality is a meaningful predictor of congestion: edges whose endpoints lie on many shortest paths between node pairs tend to accumulate more traffic. The correlation is moderate rather than strong because betweenness centrality is a node-level metric averaged to edges, and because the base model’s jam threshold creates non-linear congestion effects (once an edge is jammed, cars pile up behind it, inflating occupancy beyond what betweenness alone would predict). The outlier edge visible at the top of the plot represents a single road segment at a critical chokepoint where many shortest paths converge, producing occupancy far above the mean-field prediction.

The 95% confidence intervals (shaded bands) are narrow for low-betweenness edges (where data is dense) and widen at the high-betweenness tail (where few edges exist), reflecting the heterogeneous distribution of centrality across the network.

6 Extended Model

6.1 Driver Speed Variability

In the updated model, each car is assigned a persistent `speed_factor` on creation, drawn from a normal distribution with mean 0.9 and standard deviation 0.1, clipped to the range $[0.6, 1.1]$. The mean of 0.9 centres most drivers slightly below the speed limit, reflecting an empirical tendency for average traffic speed to sit below the posted limit. The clip prevents unrealistic extremes, such as a car moving at 20% of the limit or 50% above it.

At each time step we also apply a per-step fluctuation drawn from the absolute value of $\mathcal{N}(0, 0.15)$. Taking the absolute value produces a half-normal distribution that is always non-negative and less than 1, ensuring the perturbation only reduces speed below the

driver’s baseline. This models momentary hesitation, gear changes, and micro-adjustments that cause real drivers to vary speed from step to step. The effective driver speed at each step is:

$$\text{driver_speed} = \text{speed_limit} \times \text{speed_factor} \times (1 - \text{step_variation}) \quad (5)$$

6.2 Speed Bumps

The base model treats every road segment as a uniform surface. Real residential streets frequently have speed bumps that force drivers to decelerate regardless of the posted limit. During network initialisation, each edge is assigned a speed bump (present or absent) via a Bernoulli draw whose probability depends on the OSM highway tag: living streets have a probability of 0.5, residential streets 0.3, tertiary roads 0.1, secondary roads 0.05, and primary roads and motorways 0.0.

When a speed bump is present it reduces the car’s effective speed by 15%, with a floor of 0.3 to prevent complete stoppage:

$$\text{bump_factor} = \max(0.3, 1 - 0.15 \times \text{speed_bumps}) \quad (6)$$

where `speed_bumps` is 0 or 1. The probability table is stored as a class-level dictionary on `RoadNetwork` keyed by highway type, so it can be tuned without modifying the method body.

6.3 Pedestrian Events

The base model also ignores interactions between cars and non-vehicle road users. In dense urban areas, pedestrians stepping onto the street force drivers to brake, introducing a stochastic delay that varies by road type. Each edge is assigned a `pedestrian_prob` attribute during network initialisation keyed by OSM highway tag: residential streets receive a probability of 0.15, living streets 0.20, tertiary roads 0.08, secondary roads 0.04, primary roads 0.01, and motorways 0.0.

At every simulation step, each edge independently draws a Bernoulli trial with probability `pedestrian_prob`. If the event occurs, the car’s speed is multiplied by a uniform draw from $[0.3, 0.7]$, representing a brief but significant slowdown. Because the draw occurs at every step, the same edge may experience a pedestrian event in consecutive steps or go several steps without one, capturing the irregular nature of pedestrian interference.

6.4 Density-Based Congestion Slowdown

Real traffic slows gradually as density increases, and the effect depends on both the number of cars and the physical length of the road. A 200-metre road with 5 cars is less congested than a 50-metre road with 5 cars. The base model’s 5-car threshold is not relative to road length, which is the core problem. When multiple cars occupy the same edge, we compute density as cars per metre, and a density factor smoothly reduces speed as the edge fills, with a floor of 0.05 to guarantee that even heavily congested edges allow slow crawling rather than complete gridlock:

$$\text{density_factor} = \max\left(0.05, \frac{1}{1 + \text{density} \times 100}\right) \quad (7)$$

For example, a single car on an edge moves at full speed (`density_factor = 1.0`), and ten cars on a 100-metre road (`density = 0.1`) reduce their speed to about 9% of free-flow speed.

6.5 Continuous Speed Model with Distance Accumulation

The base model moved every car exactly one graph node per step, regardless of edge length. The updated model simulates car movement as continuous distance accumulation, where each time step represents dt minutes (default 0.5). A car’s effective speed each step starts from the edge’s posted speed limit and is successively multiplied by each modifier:

$$\begin{aligned}
 \text{speed} &= \text{speed_limit} \text{ (m/min)} \\
 \text{speed} &*= \text{speed_factor} \times (1 - \text{step_variation}) \\
 \text{speed} &*= \text{density_factor} \\
 \text{speed} &*= \text{bump_factor} \\
 \text{speed} &*= \text{ped_factor} \\
 \text{distance} &= \text{speed} \times dt
 \end{aligned} \tag{8}$$

The distance covered at each step is added to a running total (`distance_on_edge`) for each car. When this total equals or exceeds the current edge’s length, the car advances to the next node and the surplus distance carries over to the following edge. A car may cross multiple short edges in a single step if its speed is high enough, or spend many steps on a single long edge if congestion is severe.

6.6 Google Routes Data Pipeline

We obtained real-world traffic data from the Google Routes API for five time slots (8 AM, 12 PM, 3 PM, 6 PM, 10 PM). For each edge in the network, the API returns both a static (free-flow) duration and a traffic-aware duration. The ratio of these two values gives a congestion ratio per edge per time slot. A ratio of 1.0 indicates free flow; higher ratios indicate increasing congestion.

In the updated model, vehicle counts are estimated by inverting the BPR (Bureau of Public Roads) congestion formula. Given a congestion ratio $r > 1$, the volume-to-capacity ratio is recovered as:

$$\text{vc_ratio} = \left(\frac{r - 1}{0.15} \right)^{0.25} \tag{9}$$

This is then multiplied by the edge’s capacity to estimate vehicle count. Capacity is computed per edge from a base-capacity-per-lane table keyed by OSM highway type (motorway: 1800, primary: 900, secondary: 600, tertiary: 450, residential: 300) multiplied by the number of lanes. API results are cached to disk (`traffic_data_cache.pkl`) to avoid repeated queries.

6.7 BPR-Based Routing and Adaptive Rerouting

Initial edge travel times are set using the BPR formula seeded with Google-estimated vehicle counts:

$$\text{travel_time} = \text{base_time} \times \left(1 + 0.15 \times \left(\frac{\text{volume}}{\text{capacity}} \right)^4 \right) \quad (10)$$

where `base_time` is the free-flow travel time derived from the edge’s speed limit and length. Cars compute shortest paths using these travel-time weights via NetworkX.

Every `reroute_interval` steps (default 5), edge travel times are recalculated using a blend of current simulated edge occupancy and 50% of the Google baseline occupancy, then all non-arrived cars recompute their shortest paths. If the new route’s next node differs from the old one, the car’s accumulated distance resets for the new edge; otherwise, progress is preserved. This creates a feedback loop: congestion raises travel time, which diverts future cars, which reduces congestion, mirroring real-world route adaptation by navigation apps.

6.8 Traffic-Informed Car Spawning

Car origins and destinations are sampled proportionally to the Google Routes congestion ratios on adjacent edges, to better reflect real-world traffic demand. Each node is scored by the sum of excess congestion ratio ($r - 1$, where r is the Google congestion ratio) across all edges connected to it. Nodes at congested intersections receive proportionally higher weight in both origin and destination draws, so cars naturally route through the most congested corridors. Arrived non-user cars are immediately respawned with new congestion-weighted origins and destinations to maintain constant traffic flow throughout the simulation.

6.9 How These Mechanisms Interact

All mechanisms operate simultaneously during each simulation step. The sequence within a single step is:

1. At rerouting intervals, edge travel times are recalculated using the BPR formula incorporating current simulated volumes blended with Google baseline data. Cars then recompute their shortest paths using the updated weights, and distance progress is preserved unless the route actually changes.
2. For each car, the effective speed is computed by starting from the speed limit and successively applying driver variability, congestion density, speed bumps, and any pedestrian event as multiplicative factors.
3. The resulting distance is accumulated. When it exceeds the edge length, the car advances to the next node, carrying surplus distance forward.

Routing decisions (step 1) respond to congestion through updated edge weights, while movement speed (step 2) is modulated by individual driver characteristics, road infrastructure, and local traffic density. A slow driver on a bumpy, pedestrian-heavy residential street experiences compounding delays, while a fast driver on a clear primary road moves at nearly free-flow speed. This layered structure produces emergent traffic patterns that are more heterogeneous than the original base model.

6.10 Betweenness Centrality in the Extended Model

To understand whether the extended model’s congestion patterns are still driven by network topology, we repeat the betweenness centrality analysis from the base model. We compute node-level betweenness centrality on the extended model’s graph using BPR-adjusted

travel-time weights, average endpoint centralities to produce per-edge scores, and plot these against average edge occupancy from 50 independent simulation runs with 100 cars each (Figure 6).

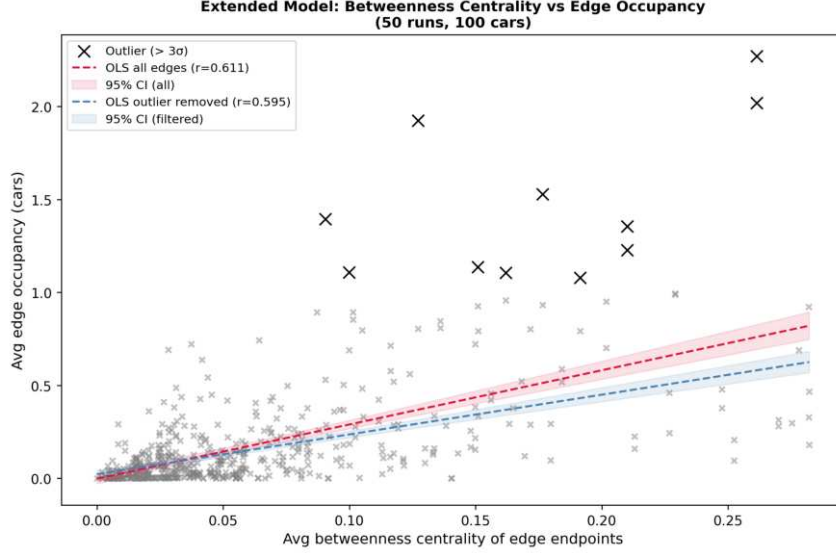


Figure 6: Extended model: betweenness centrality vs. average edge occupancy (100 cars, 50 runs) with OLS regression lines and 95% CIs.

The extended model shows a notably stronger correlation between betweenness centrality and edge occupancy than the base model: $r \approx 0.61$ with all edges included and $r \approx 0.57$ with outliers removed, compared to $r \approx 0.30$ and $r \approx 0.36$ in the base model. Doubling the correlation strength is a direct consequence of the extended model’s BPR-based travel times making edge weights responsive to volume: as congestion builds on high-centrality edges and their travel times increase, the adaptive rerouting mechanism diverts some cars to alternative routes. The feedback loop means that high-centrality edges still attract disproportionate traffic (they are the natural shortest paths), but congestion is partially redistributed by rerouting. The result is a tighter, more linear relationship between centrality and occupancy, with fewer extreme outliers, because rerouting prevents any single edge from becoming catastrophically overloaded.

Interestingly, the filtered correlation ($r \approx 0.57$) is slightly lower than the all-edges correlation ($r \approx 0.61$), the opposite of the base model pattern. This suggests that the extended model’s outlier edges are well-predicted by betweenness centrality: the highest-occupancy edges are genuinely the most central ones, rather than artifacts of stochastic pile-up as in the base model. The 95% confidence intervals are also tighter than in the base model, reflecting the more regular relationship between structural importance and traffic load when congestion-responsive routing is active.

7 Estimating Realistic Car Counts via Little’s Law

To determine how many cars are physically present on the road network at any given instant, we apply Little’s Law from queueing theory. The Google Routes data provides an estimated hourly vehicle throughput λ_e for each edge e (derived from the inverse-BPR

formula), but this is a flow rate, not a count of cars simultaneously on the road. To convert flow to concurrent occupancy:

$$L = \sum_{e \in E} \lambda_e \cdot W_e \quad (11)$$

where L is the total number of cars present on the network, λ_e is the estimated vehicles per hour on edge e , and W_e is the time in hours to traverse edge e at its posted speed limit:

$$W_e = \frac{\ell_e}{v_e} \quad (12)$$

where ℓ_e is the edge length in km and v_e is the speed limit in km/h. Summing across all edges gives the expected number of cars physically occupying the 1 km-radius network at any instant during the 6 PM rush hour. This estimate accounts for the fact that short, slow edges hold cars for longer than long, fast ones, producing a more realistic count than simply summing volumes.

8 Google Routes API Ground Truth

Before comparing the two models, we first visualise the ground-truth congestion data obtained from the Google Routes API for the 6 PM rush hour (Figure 7). For each edge in the network, Google returns a static (free-flow) duration and a traffic-aware duration. Their ratio, the congestion ratio $r = t_{\text{traffic}}/t_{\text{free-flow}}$, quantifies how much slower traffic moves compared to free-flow conditions. A ratio of 1.0 means the road is uncongested; values above 1.0 indicate increasing delay.



Figure 7: Google Routes API congestion heatmap at 6 PM. Green = free-flow ($r \approx 1$); red = highest delay.

The Google congestion heatmap reveals a non-trivial spatial pattern. Congestion ratios range from ~ 0.8 (below free-flow, indicating Google’s static estimate is conservative on some edges) up to ~ 2.0 on the most delayed segments. The highest-ratio edges (red, $r \approx 2.0$) are concentrated in the western and upper portions of the network, corresponding to primary roads that serve as through-routes for cross-neighbourhood traffic. A broad band of yellow edges ($r \approx 1.2\text{--}1.5$) covers much of the central and southern network, indicating moderate congestion on secondary and tertiary streets during the rush hour. Residential side streets in the eastern periphery remain green ($r \approx 1.0$), near free-flow. This spatial heterogeneity (a few high-delay corridors embedded in a network of moderately congested streets) serves as the reference against which we evaluate both models.

9 Base vs. Extended Model Comparison

To evaluate how much the extended model improves over the base model, we run both at the Little’s Law estimated car count ($N = 852$ cars physically present on the network during the 6 PM rush hour) and compare their congestion patterns against the Google Routes traffic data for the same time slot.

9.1 Congestion Heatmap Comparison

For each model we compute an average per-edge congestion metric, normalise it to $[0, 1]$ via min-max scaling, and plot it on the road network using a shared red–yellow–green colour map (red = most congested, green = least congested). The three heatmaps are displayed side by side in Figure 8.

For the base model, we run 50 independent Monte Carlo simulations with different random seeds (varying car origins and destinations), each running until all cars have reached their destinations. At every step of every run we record the occupancy of each edge, then average occupancy across all runs and steps and apply min-max normalisation.

For the extended model, we run a single simulation with seed 42, using BPR-initialised travel times, congestion-weighted car spawning, and adaptive rerouting every 5 steps. The simulation runs until the user car arrives at its destination. The average per-edge car density (averaged across all simulation frames) is then normalised.

For the Google reference, the congestion ratio $r = t_{\text{traffic}}/t_{\text{free-flow}}$ from the Google Routes API at 6 PM is used directly as the per-edge metric and normalised to $[0, 1]$.



Figure 8: Congestion heatmaps at $N = 852$ cars. Left: base model. Centre: extended model. Right: Google ground truth. All min-max normalised.

The base model’s congestion pattern (left panel) is governed almost entirely by network topology: the vast majority of edges are green with only a small number of isolated edges showing red or orange. These hotspots correspond to high-betweenness-centrality chokepoints, edges that lie on many shortest paths between randomly sampled origin–destination pairs. Because car spawning is uniform and paths are computed once without rerouting, the base model produces a sparse, concentrated congestion pattern: a few bottleneck edges are severely loaded while the rest of the network is nearly empty. The pattern bears little resemblance to the Google reference, where congestion is distributed across many corridors. Several primary roads that Google identifies as heavily congested ($r > 1.5$) appear green in the base model, and conversely the base model’s hotspots do not correspond to Google’s congested corridors. That said, the model does reveal which roads are topologically most congested and shows how those relate to the Google map.

The extended model (centre panel) produces a qualitatively different congestion pattern with a few prominent red and orange edges visible across the network, particularly along north–south and east–west corridors. However, the spatial pattern is not obviously closer to the Google reference than the base model’s sparse pattern. Where Google shows a diffuse, widespread band of yellow and light-green congestion spread across many streets, the extended model concentrates congestion onto a handful of deep-red edges while leaving the rest of the network green. The extended model has shifted the bottlenecks to different locations compared to the base model but has not reproduced Google’s characteristic feature of moderate congestion distributed across many roads simultaneously.

A few reasons likely explain this. First, the base model averages over 50 independent Monte Carlo runs, smoothing stochastic noise into a stable average congestion pattern, whereas the extended model runs a single simulation (seed 42) that terminates when the user car arrives, capturing one particular realisation of traffic rather than a long-run average.

Second, the BPR formula’s fourth-power term amplifies congestion non-linearly: edges that accumulate even slightly more traffic than neighbours get disproportionately higher travel times, which then diverts cars away via rerouting and creates a winner-takes-all dynamic where a few edges end up deep red while everything else stays green. Third, the congestion-weighted car spawning creates a positive feedback loop: cars are spawned near congested edges, making those edges more congested, which attracts even more spawns. This self-reinforcing cycle over-concentrates traffic in a small number of corridors rather than spreading it out.

9.2 Per-Edge Scatter Plot Analysis

To move beyond visual comparison and quantify the edge-level agreement between each model and Google, we construct normalised congestion vectors and produce scatter plots (Figure 9). For each edge e , let $g_e \in [0, 1]$ be the min-max normalised Google congestion ratio and let $s_e \in [0, 1]$ be the corresponding normalised simulated value (average occupancy for the base model, average density for the extended model). Each point in the scatter plot represents one edge, with g_e on the horizontal axis and s_e on the vertical axis. A model that perfectly reproduces Google’s congestion pattern would place all points on the $y = x$ diagonal.

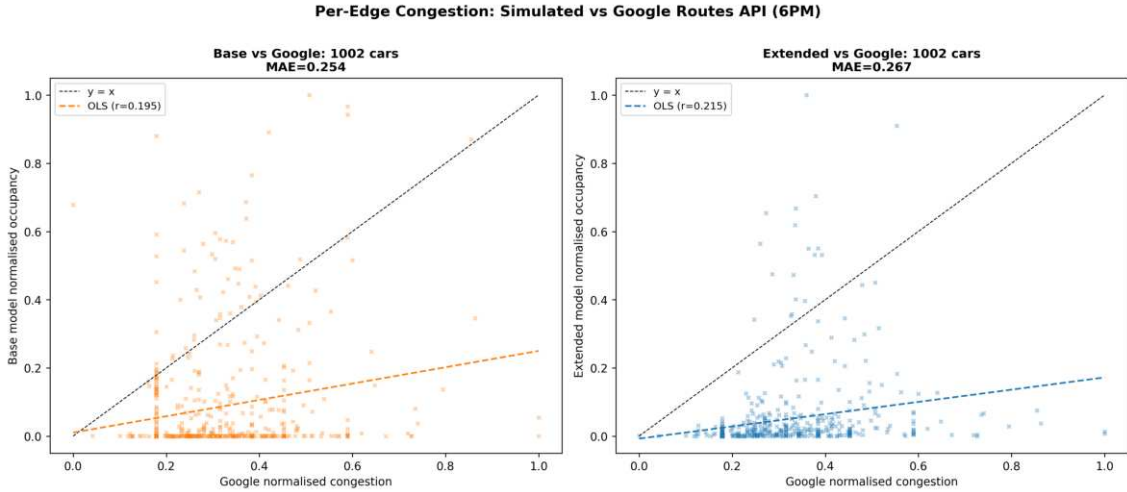


Figure 9: Per-edge simulated vs. Google congestion at $N = 852$ cars. Left: base model. Right: extended model. Dashed black = $y = x$; coloured = OLS fit.

We compute two summary statistics for each model. The Pearson correlation coefficient r measures the linear association between the simulated and Google congestion vectors. Mean absolute error (MAE) is defined as:

$$\text{MAE} = \frac{1}{|E|} \sum_{e \in E} |s_e - g_e| \quad (13)$$

measuring the average magnitude of per-edge deviation from Google’s congestion values.

The base model achieves $r = 0.213$ and $\text{MAE} = 0.258$. The OLS regression line has a shallow positive slope, indicating that the base model’s normalised occupancy responds

only weakly to Google’s congestion signal. The scatter is wide: many edges that Google identifies as heavily congested ($g_e > 0.5$) receive near-zero simulated occupancy ($s_e < 0.1$), and a cluster of high-occupancy base-model edges sits at low Google values ($g_e \approx 0.15\text{--}0.25$), corresponding to topological bottlenecks that carry heavy shortest-path traffic regardless of real-world demand.

The extended model achieves $r = 0.219$ and $\text{MAE} = 0.261$, essentially indistinguishable from the base model in terms of aggregate per-edge correlation. Despite the qualitatively different spatial pattern visible in the heatmaps, the edge-level scatter reveals that both models explain only $\sim 4\text{--}5\%$ of the variance in Google’s congestion signal ($R^2 \approx 0.045$). The extended model’s OLS line has a marginally steeper slope, indicating slightly better responsiveness to Google’s signal in the high-congestion tail, but the improvement is not statistically meaningful at the per-edge level.

The core lesson here is that adding realism to a simulation does not automatically improve its agreement with ground truth if the underlying demand model is too coarse. The extended model’s mechanisms are individually well-motivated (drivers do vary in speed, roads do have bumps, pedestrians do cause delays) but without fine-grained origin and destination demand data, the simulation cannot know which specific edges should carry traffic.